18 种版式 / 5 种图表 / Typst 原生排版 / 6 套主题

幻述 Huanshu

01

一页 Markdown，一副 keynote —— 18 种版式、5 种图表
与原生公式排版的生产级幻灯流水线

本场导览

01

问题与愿景

手工排版为什么难以规模化，以及我们要守住什么。

02

流水线解剖

规划、生成、校验、渲染四段的职责边界。

03

实测数据

耗时、探测次数、缓存命中与时间线。

04

取舍与下一步

SWOT 分析与已经落地的四件事。

从一份 2 800 字的 Markdown 源稿出发，看一条可复现的流水线如何决定版式、补齐内容，并把每一页排到刚好放下。

章节导览

从文档到演示

本章关注

4 个子主题

> 第一章节要回答的问题：为什么「把内容写对」和「把版式排好」必须分成两件事。

01

内容先行

源稿只描述事实与结构：谁在什么时候、交付了什么、数据是多少。

02

版式后置

版式在整副牌组上分配，而不是逐页贪心，避免相邻撞版。

03

渲染兜底

任何一页都必须排得下：先实测，再缩放，最后才裁切并上报。

04

可回归

同一份源稿固定后，主题与版式的每次调整都能逐页对比。

四项能力，构成一条完整的交付链路

从解析到渲染，每一步都有明确的失败边界

结构化解析

输入层

把标题层级、正文、图表围栏与公式 拆成可校验的结构。

- H2/H3 决定分页
- 图表围栏绑定小节
- 公式原样保护

→ 解析阶段就把错误报给作者

Parser

全局版式分配

决策层

每页先出 2–3 个候选并打分，再在整副牌组上求一次全局最优。

- 相邻撞版扣分
- 单一版式设上限
- 超宽比例动态放宽

→ 分配结果与并发数无关

Planner

原生矢量渲染

渲染层

Typst 直接排版出 PDF 与位图，图表、公式、图标全部由代码绘制。

- 内容实测缩放
- 坏图不拖垮整副
- 五种图表齐全

→ 屏幕与导出共用同一条渲染路径

Renderer

自适应与回归

质量层

字号按内容量分档，超出可用区域会被裁切并逐页上报。

- 探测次数可观测
- 溢出页逐条上报
- 主题切换逐页对比

→ 把「排不下」变成一份可读的报告

Quality

四张卡片从左到右依次对应输入、决策、渲染与质量四个环节。

手工做幻灯的四次断点

⚠ 版式靠手感

同一份内容，不同人排出来的信息密度完全不同，评审时只能争论「感觉」。

标准不统一　复用成本高　对齐困难

内容与版式耦合

写作阶段就要决定分栏与层级，改一句话常常推翻整页。

返工频繁　结构走形　批量修改难

图表与公式外挂

截图贴进来的图表不跟随主题，也无法随数据更新。

分辨率不可控　配色难统一　数据难追溯

回归全靠肉眼

改了主题或组件，只能逐页翻看有没有溢出和错位。

问题发现晚　无法自动比对　改版风险高

四张卡片正好对应流水线里被自动化的四个环节。

手工排版 vs 幻述流水线

同一份 2 800 字源稿，两条路径的成本结构

	优势 幻述流水线	±	劣势 手工排版
01	★ 内容与版式解耦 写作时只关心结构与事实	01	★ 边写边排版 内容一改就要重排
02	✓ 版式全局分配 28 页只撞版 3 次，换主题可重排	02	× 逐页凭手感 信息密度随当下状态波动
03	✓ 图表与公式原生 跟随主题换色，数据可追溯	03	× 图表靠截图 换主题要重新导出所有图
04	✓ 溢出即上报 裁切页在导出报告里逐条列出	04	× 靠肉眼回归 溢出往往在投影时才被发现
	把「每页都一样好」交给规则，而不是状态		质量取决于当下状态，无法稳定复现

高亮项是差异最大的一环：内容与版式的解耦程度。

SUMMARY

让每一页自己找到版式
版式不是作者的选择题：候选打分加上全局分配，会在约束内挑出最合适的一种。



候选 2–3 个

模型只在当页可用的版式里打分。



全局一次求解

相邻同款扣 3 分，单一版式不超 30%。



规则表过滤

比例、要点数量、素材有无都是硬条件。



兜底永远可用

模型失败时按先验分排序，仍给出合法页。

这是流水线里唯一需要「整副牌组视角」的一步。

章节导览

流水线解剖

本章关注

6 个子主题

> 第二章节逐段拆开生成过程：规划决定版面，生成填充内容，校验守住边界，渲染负责放下。

01

分页规划

H2 决定章节号，H3 与小节内容量决定分页位置。

02

候选打分

每页仅在本页可用版式里排序，模型不做多样性判断。

03

全局分配

相邻项精确求解，全局项用单页交换逼近。

04

内容生成

按选定版式的字段填充，缺失字段先补默认值。

05

校验修复

结构错误带反馈重试，最后一轮退回通用版式。

06

排版交付

Typst 一次编译出 PDF，同一份蓝图光栅出缩略图。

三段式流水线

规划 → 生成 → 校验，每一段都有确定性兜底

Phase 1 规划与分配

关键动作 03

1. **解析源稿**
   H2/H3 决定分页与章节号
2. **候选打分**
   只在本页可用版式里排序
3. **全局分配**
   相邻项用维特比精确求解

✓ 分配结果与并发数无关

Phase 2 内容生成

关键动作 03

1. **按版式填充**
   字段不足时按结构补齐
2. **规范化**
   长度收敛、默认值修复
3. **图表挂载**
   数据由源稿原样搬入

✓ 图表数据绝不由模型改写

Phase 3 校验与修复

关键动作 03

1. **结构校验**
   缺字段、越界枚举立即拦下
2. **带反馈重试**
   把失败原因回灌给模型
3. **兜底页**
   仍失败则退回通用版式

✓ 任何一页都不会静默丢失

并发生成只影响速度，不影响版式分配的结果。

从源稿到出图：四步

每一步的产物都能单独检查

1



解析源稿

切分章节、正文、图表围栏与公式。

2



分配版式

每页候选打分，整副牌组一次求解。

3



生成内容

按版式字段填充，规范化后校验。

4



排版交付

Typst 出 PDF 与位图，溢出逐页上报。

失败重试只回到出问题的那一步，不会从头再来。

四层结构，每层只管一件事

自下而上：输入 → 决策 → 渲染 → 交付

01

输入层只描述事实

01

源稿里的标题层级与图表数据是唯一事实来源，不含任何版式信息。

02

决策层只认规则

02

候选过滤、先验分与相邻惩罚都写在规则表里，可测试、可回归。

03

渲染层只管放下

03

字号按内容量分档，实测后缩放；仍放不下就裁切并上报。

架构层次

交付层 — PDF · PNG · 演示位图

↑

渲染层 — Typst 版式与图表

↑

决策层 — 候选打分 + 全局分配

↑

输入层 — Markdown 源稿

层与层之间的接口就是蓝图 JSON；换掉任意一层，其余不动。

版式分配是一次全局优化

以 28 页牌组为例：先算分，再算重复代价

先解相邻项

相邻同款的惩罚只依赖前一项，状态空间是线性的。

再放宽全局项

单一版式上限用单页交换逼近，跑完仍是确定性结果。

最后写入调试目录

逐页候选、被改选的页与兜底页都落盘，方便事后归因。

目标函数

把每页的候选分数加总，再对重复版式收取代价：

$$S = \sum_{i} s_i - 3 \cdot \sum_{\text{相邻同款}} 1 - 1 \cdot \sum_{\text{隔页同款}} 1$$

约束是任一版式不超过内容页的 30%（至少 2 页）。相邻项用维特比精确求解，全局项再用单页交换做局部搜索。

正确的关注顺序：

1 候选分数 s_i （模型按本页适配度打分）

2 相邻同款惩罚 （权重 3，精确求解）

3 隔页同款惩罚 （权重 1，抑制交替撞版）

4 超出上限的次数 （权重 2，局部搜索）

交付前的四道检查

任何一条不通过，导出报告都会列出具体页码

01 **结构完整**

每个版式的必填字段都在，枚举值都落在允许集合内。

02 **图表可读**

五种图表的数据行都能画出，环形图至少有一个正值扇区。

03 **版面放得下**

实测高度超过可用区域时缩放，仍溢出则裁切并上报。

04 **素材齐备**

引用的图片全部解析成功，坏图只影响它自己那一页。

产品界面

三步工作流，一个窗口



第一步 · 准备文稿

Markdown 源稿里可以直接写图表围栏与 公式。

- ✓ H2 决定章节
- ✓ 图表绑定小节
- ✓ 公式原样保护

源稿是唯一事实来源

第二步 · 生成看板

解析与规划

↓

逐页生成

↓

校验与修复

↓

缩略图预览

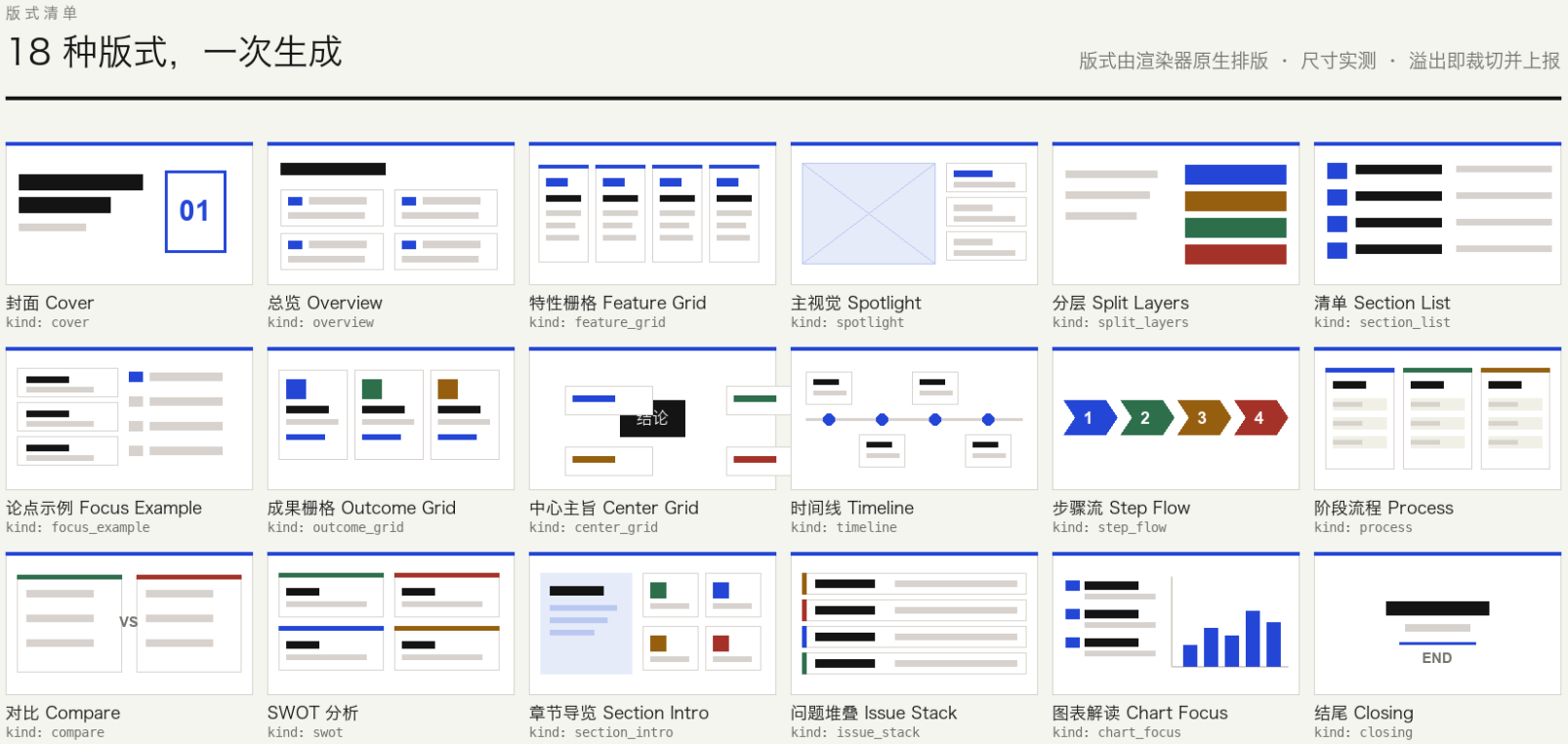
第三步 · 编辑器

逐字段修改并实时预览，图表先过 JSON 校验再写回。

- ✓ 单页重试
- ✓ 图表 JSON 校验

预览与导出同一条渲染路径

版式清单 18 种版式，覆盖常见表达



版式从哪来

每页先出候选，再在整副牌组上做一次全局分配。

- ✓ 相邻同款惩罚 3
- ✓ 单一版式上限 30%

分配结果与并发数无关

边界怎么处理

超宽比例会屏蔽不适合的版式，要点数量也参与过滤。

- ✓ 32:9 禁用主视觉
- ✓ 48:9 再禁对比与示例

通用兜底版式永远可用

生成流水线

八个环节，一条可观测链路

生成流水线

从一页 Markdown 到一副 keynote



确定性兜底：LLM 未给出可用候选时，按先验分表排序并过滤边界条件，仍能给出合法版式。

确定性兜底

模型没给出可用候选时，按先验分表排序并过滤硬条件。

- ✓ 先验分覆盖全部版式
- ✓ 命中的信号写进调试目录

可观测

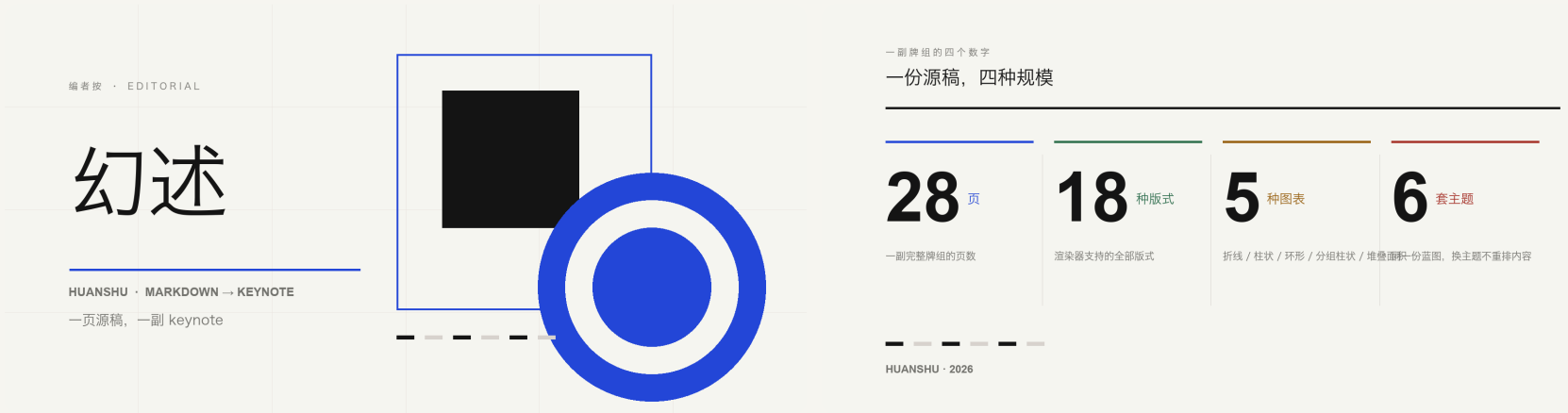
每次生成都会把候选、分配与最终版式写进调试目录。

- ✓ 逐页候选 JSON
- ✓ 分配对比 JSON
- ✓ 最终版式 JSON

问题可以精确归因到某一页

牌组数字

一份源稿，四种规模



同一份蓝图

页数、版式、图表与主题都是蓝图的属性，不写死在模板里。

- ✓ 换主题不重排内容
- ✓ 换画幅不重写字段

28 页一次生成

多图平铺

屏幕上这是轮播；PDF 一页放不下动画，于是改为等分平铺，最多六张。

- ✓ 超过六张显示 +N
- ✓ 缺图只影响自己这一格

章节导览

实测数据

本章关注

4 个子主题

> 第三章节给出同一台机器、同一份源稿上的实测结果：耗时、探测次数与缓存收益。

01

端到端耗时

六个月里从 412 秒降到 121 秒，主要来自并发与增量编译。

02

单页首帧

最重的图片页约 168 ms，最轻的封面页只需 78 ms。

03

探测次数

自适应缩放在可行边界上插值，最坏情况 9 次探测。

04

缓存收益

重复导出时大部分页直接命中记忆化缓存，不再全量重排。

端到端耗时：六个月下降 71%

一副 28 页牌组，从源稿到 PDF 的总耗时（秒）

01 并发规划

每页候选打分并发执行，规划阶段从 180 秒降到 42 秒。

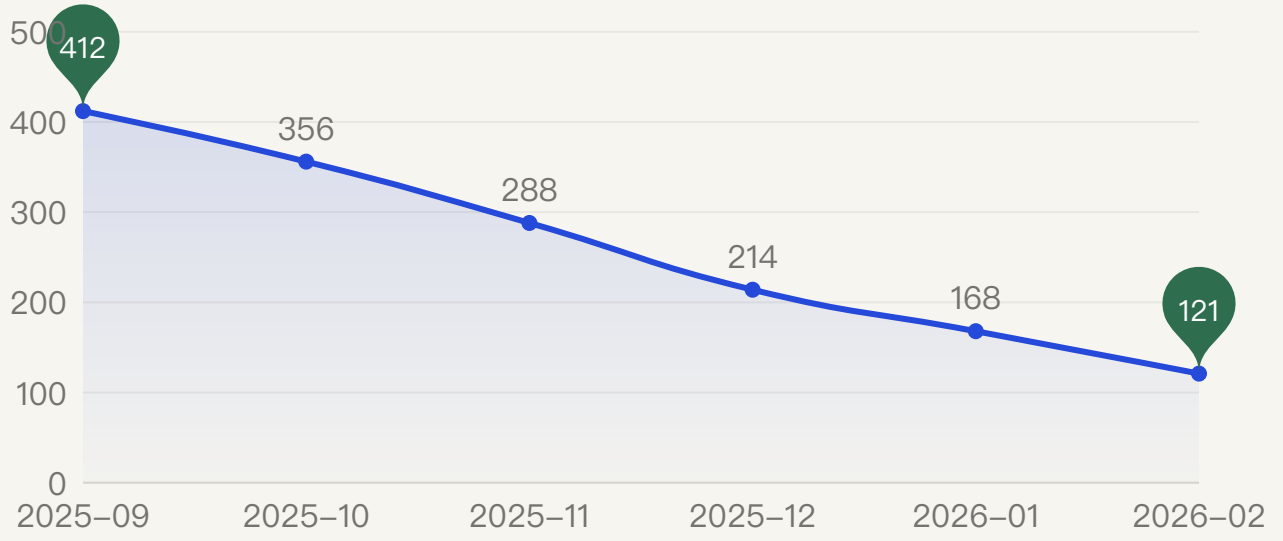
02 增量编译

只重编变化的页，重复导出几乎不再付全量成本。

03 下一步

目标是把中位耗时压到 90 秒以内。

端到端生成耗时（秒）



数据来源：幻述基准测试 · 28 页牌组 · 2026–02

同一台机器、同一份 2 800 字源稿，取五次运行的中位数。

单页首帧：版式之间差 2 倍

同样的字体与主题，渲染器实测（毫秒）

01 图片页最重

主视觉页要解码图片并实测侧栏高度，168 ms 居首。

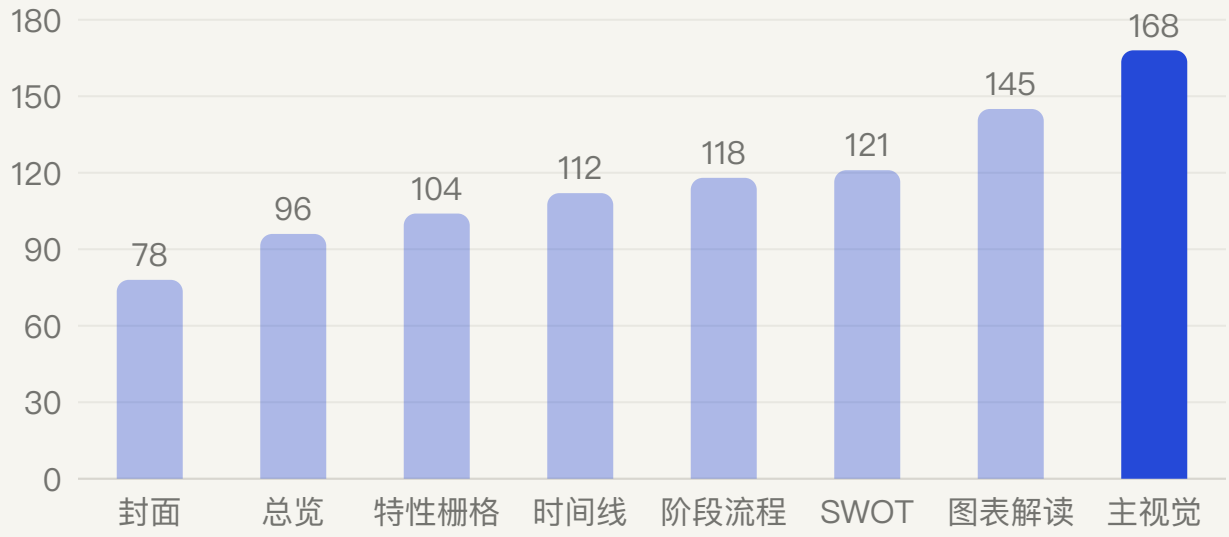
02 图表页次之

五种图表都由 Typst 现画，约 145 ms。

03 栅格页最轻

特性栅格只需排四张卡片，104 ms 就能出首帧。

各版式单页首帧耗时（毫秒）



数据来源：幻述基准测试 · 28 页牌组 · 2026-02

release 构建，560 px 宽位图，首次渲染（不含字体扫描）。

渲染耗时都花在哪

文字整形仍然是大头

01 文字整形

中英混排的换行与字距占掉约 41% 的时间。

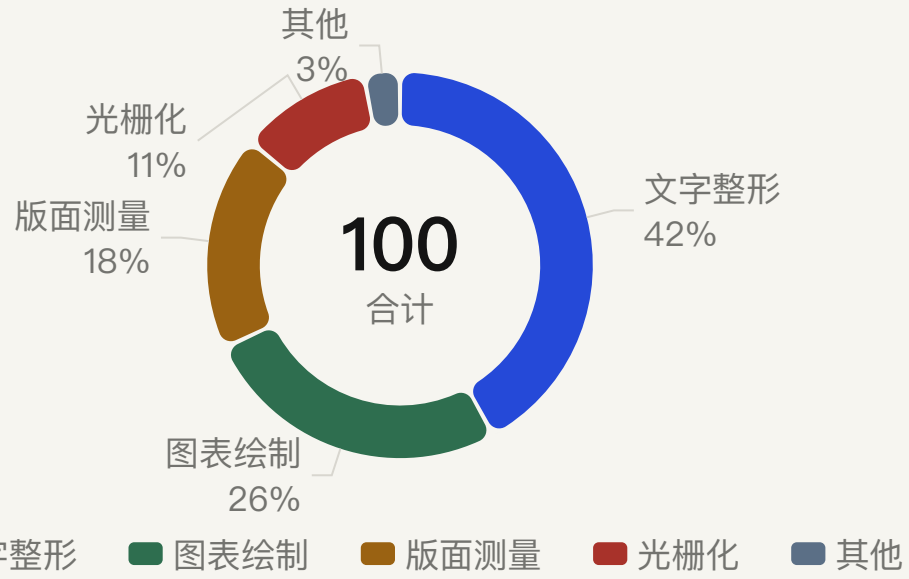
02 图表绘制

五种图表现画，约占四分之一。

03 版面测量

自适应缩放需要反复试排，测量成本不可忽略。

单页渲染耗时构成



数据来源：幻述基准测试 · 28 页牌组 · 2026-02

对 28 页逐页统计后按阶段归类，单位毫秒。

三种画幅下的最坏探测次数

自适应缩放要在可行边界上反复试排

01 画幅越宽

超宽画幅的列更多、卡片更窄，可用高度反而更紧张。

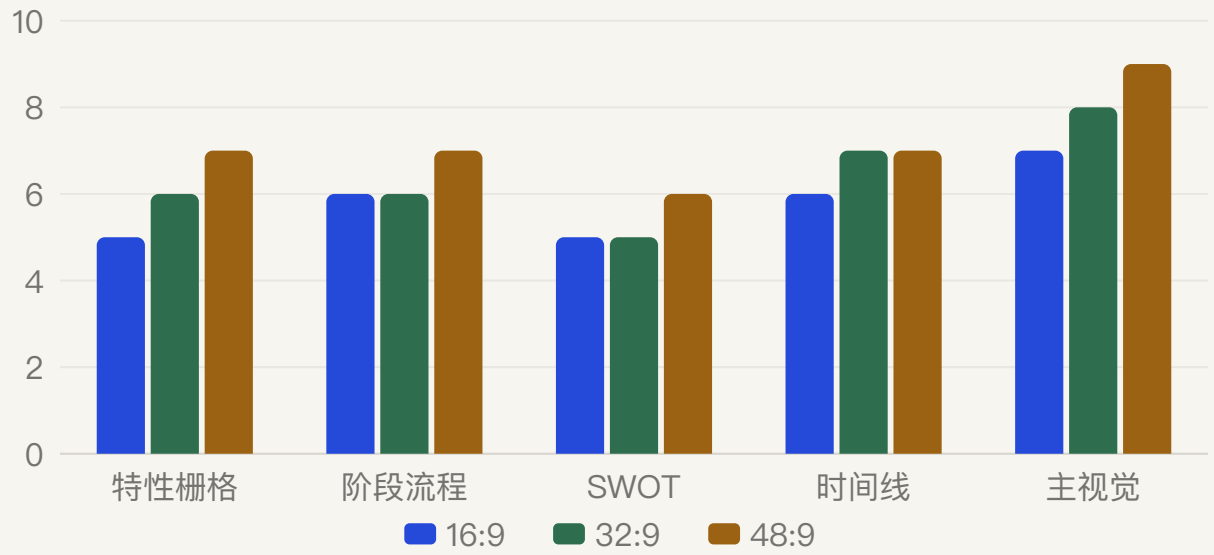
02 主视觉最贵

侧栏与图片都要实测，48:9 下最坏要探测 9 次。

03 上界可控

改动搜索策略后，探测次数比二分版少 3–5 次。

最坏情况下的字号探测次数



数据来源：幻述基准测试 · 28 页牌组 · 2026-02

探测次数越多，代表这一页在可用面积上越接近临界。

缓存命中撑起的增量编译

累计编译次数（千次），阴影部分是重复排版

01 命中率上升

编辑器只发当前页，重复内容直接命中记忆化缓存。

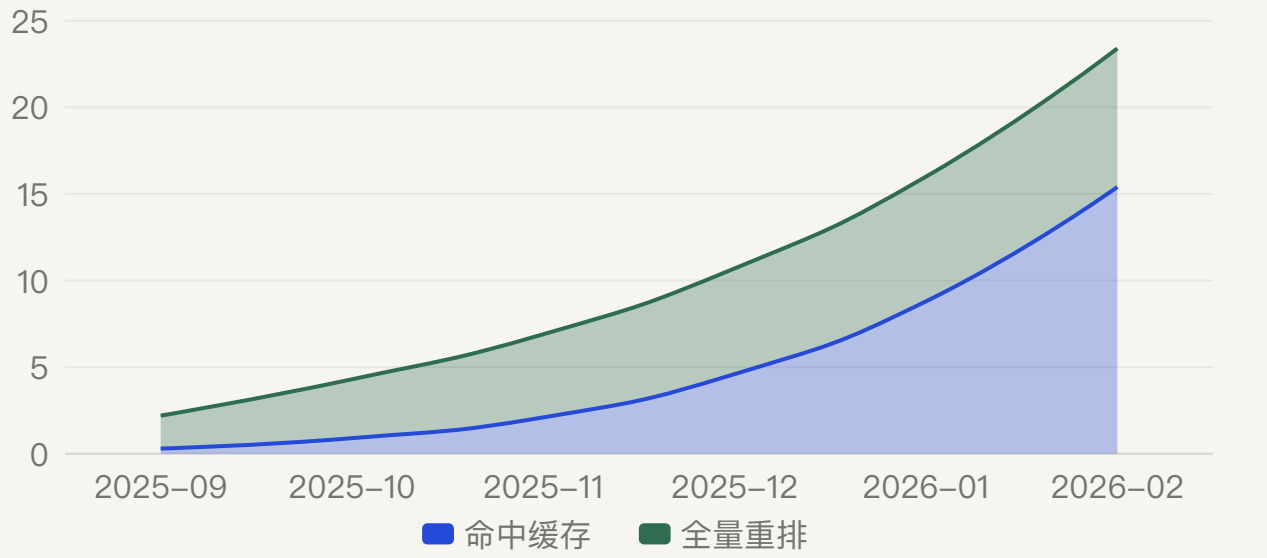
02 成本可预测

增量编译让每次改动的代价与改动页数成正比。

03 回收策略

编译后主动回收缓存，避免长时间运行内存只增不减。

累计编译次数（千次）



数据来源：幻述基准测试 · 28 页牌组 · 2026–02

统计口径是所有渲染入口（缩略图、演示位图、PDF 导出）之和。

从原型到流水线

五个关键节点，每一步都留下了可回归的固定样本

第一版生成流水线

从 Markdown 到蓝图跑通，版式仍由逐页贪心决定。

2025-10-16

2025-12-04

全局版式分配

候选打分、维特比与单页交换，相邻撞版显著下降。

Typst 原生渲染

缩略图、演示位图与 PDF 收敛到同一条渲染路径。

2026-01-20

2026-02-11

图表与公式原生

五种图表与 LaTeX 公式由代码绘制，不再依赖外挂。

自适应缩放重写

在可行边界上插值搜索，字号更贴近可用上限。

2026-03-02

固定样本是这套流水线能被持续改动的前提。

章节导览

取舍与下一步

本章关注

4 个子主题

> 最后一章节把视角放回真实工作流：哪些是优势，哪些仍是风险，以及接下来做什么。

01

守住协议

蓝图 JSON 是层与层之间唯一的契约，先保它稳定。

02

扩样本

把固定样本扩成主题与画幅的组合回归集。

03

接数据

让图表可以从真实数据源刷新，而不是手写 JSON。

04

放开迭代

模型与主题都可以换，只要样本还在，改动就有底。

把流水线放进真实工作流

优势来自确定性，风险来自协议漂移

Strengths

优势在于确定性与可复现。

- 同一份源稿每次得到同一副牌组
- 版式、图表与公式都由代码排版，换主题无需重做
- 溢出与缺图会在导出报告里逐页列出

Weaknesses

弱点集中在极端内容与人工维护。

- 长文与极端文案仍可能出现裁切
- 版式上限写在规则表里，需要人维护
- 多模态配图只参与选页，不参与裁剪

Opportunities

- 把固定样本扩成主题回归集
- 为 32:9 与 48:9 补专项样本
- 接入真实数据源，让图表随数据更新

机会在于把回归做得更系统。

Threats

- 版式协议演进会让旧牌组语义失真
- 字体缺失会让排版度量整体偏移
- 更换模型会改变候选分布，需要重新标定先验分

风险主要来自协议与环境的漂移。

先用固定样本守住协议边界，再放开模型与主题的迭代速度——只要样本还能逐页对比，任何改动都有回退的依据。

四件事已经落地

都在同一条渲染路径上实现，没有第二套兜底实现

一副牌组两种产物

同一份蓝图，屏幕上是位图，交付是矢量 PDF。

版式全覆盖

18 种版式、5 种图表、6 套主题共用同一套蓝图结构。

溢出可上报

缩放仍放不下的页会被裁切，并逐条写进导出报告。

可回归

固定样本加逐页位图，让主题与版式调整都能对比。

以上四项都能用一份固定源稿完整重放。

18 种版式 5 种图表 6 套主题 Typst 原生

END

开始写你的第一页

Markdown 进来，keynote 出去 —— 版式、图表与公式都交给渲染器。

下一步：把这份 28 页的样本存成基线，换一套主题再放一遍。

幻述